

Mutation Detection Operational Specification: Substrate-Resident Detection Rules, Coverage Depth, and Governance Workflow Triggering in CKS Deployments

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 12, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in “The Instinct/Reasoning Separation Outside the Model” (Li, April 2026), the second paper in the CKS theory series following “Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems” (Li, April 2026).

Abstract

Paper 2’s mutation governance commitment activates when a mutation event is detected. B2.61 defines what counts as a mutation event — any change to the LLM that forms the instinct layer, arriving from the vendor’s domain outside the substrate’s authority. B2.62 formalizes the operational question that follows: how do CKS deployments detect such events, and how does detection connect to the governance workflow? The answer has four parts. Detection rules are substrate-resident authoritative content authored per A2.04 and carried per A2.46; they are not ad hoc monitoring configurations. Multiple detection mechanisms provide coverage depth: LLM version monitoring is primary; vendor notification monitoring and periodic review serve as backup; behavioral drift detection covers silent weight changes within the same version label. The mutation-vs-directed-selection distinction matters at the detection layer: detection applies the B2.61 instinct-layer test, ruling out DNA-layer changes as non-mutation events. Detection events are recorded per A2.40, making each detection event the first entry in the mutation governance event chain. Detection triggers the mutation governance workflow — verification gate evaluation per B2.63, routing consideration per B2.64, pinning review per B2.65 — and detection failure means mutation integration without governance. This note formalizes mutation detection as the operational bridge between mutation event definition and governance workflow, and establishes coverage-determines-governance as the central operational consequence.

1. Why mutation detection needs standalone formalization

B1.13 establishes that mutation governance activates when a mutation event occurs. B2.61 defines mutation events as changes to the LLM at the instinct layer, arriving from upstream vendors outside the substrate’s control, classified by whether they change what the instinct layer does. Neither B1.13 nor B2.61 commits to how deployments detect such events in practice. The detection specification is the missing operational link: without detection, governance cannot activate regardless of how carefully the governance workflow is designed.

The detection specification has independent prior-art territory on several dimensions. The treatment of detection rules as substrate-resident authoritative content — authored, versioned, and governed — is a distinct architectural commitment separate from both the definition of mutation events and the governance steps that follow. The multi-mechanism coverage architecture, the mutation-vs-directed-selection distinction

applied at the detection layer, the detection event recording, and the governance workflow triggering are each individually formalizable. Leaving them bundled with B2.61's event definition or with B2.63's verification gate treatment would obscure the distinct patentable territory of the detection specification itself.

B2.62 occupies the second position in the six-note decomposition of B1.13: B2.61 (mutation event definition) — B2.62 (this note: mutation detection operational specification) — B2.63 (verification gate triggering) — B2.64 (routing adaptation) — B2.65 (pinning enforcement) — B2.66 (mutation governance verification). Within that sequence, B2.62 is the bridge: it converts the conceptual definition of mutation events into the operational detection machinery that makes governance possible.

2. The architectural specification

The mutation detection operational specification has five components.

Detection rules as substrate-resident authoritative content. Detection rules are authored per A2.04 — human-authored, not LLM-generated autonomously — and carried as authoritative substrate content per A2.46. This means detection rules are inspectable, modifiable, subject to the three governance rights under A1.01, versioned, and addressable within the substrate. Detection rules are not external monitoring configurations applied around the substrate; they are substrate content that governs how mutation events are identified. The architectural consequence is that detection is itself governed: detection rules can be updated through the standard authority architecture, improved as operational experience accumulates, and tested as substrate content rather than as external configuration.

LLM version monitoring (primary detection mechanism). Per A1.08, the substrate is the source of truth for authoritative deployment state, including the current LLM version. Version monitoring operates against this record: it detects discrepancy between the LLM version the substrate records as current and the LLM version available from the vendor (whether surfaced through a vendor API, a deployment notification, or a governance review that examines vendor communications). When a discrepancy is detected, version change classification is triggered and the mutation detection workflow fires. Version monitoring is the primary detection mechanism because it is the most direct operational expression of the A1.08 source-of-truth commitment applied to the instinct layer.

Backup detection mechanisms. Three additional mechanisms provide coverage depth beyond version monitoring. Vendor notification monitoring watches for vendor communications — model update announcements, deprecation notices, migration requirements — that signal mutation events before or independent of the substrate's version record reflecting the change. Periodic review detection uses governance-scheduled reviews to examine whether the current recorded LLM version remains current; it provides coverage where automated monitoring has structural gaps. Behavioral drift detection is the supplementary mechanism for the specific case of silent weight changes: when a vendor updates model weights within the same version label without publishing an updated version identifier, version monitoring does not detect a change. Behavioral drift detection monitors for divergence between expected and observed cell operational behavior in ways that suggest LLM behavioral change rather than DNA-layer change; when drift exceeds governed thresholds, mutation event classification is triggered. Behavioral drift detection is explicitly supplementary — it is not a substitute for version monitoring, because behavioral drift has multiple possible causes including DNA-layer changes and input pattern changes that are not mutation events.

The mutation-vs-directed-selection distinction at the detection layer. Detection applies the B2.61 instinct-layer test: does this change the instinct layer? DNA-layer changes through directed selection per B1.14 are substrate-internal changes governed through the standard authority architecture; they modify orchestration rules, cell behavior configurations, and content, but they do not change the LLM. Detection rules apply the B2.61 test explicitly: if the change is to the DNA layer, the mutation detection workflow is not triggered regardless of the magnitude of the DNA-layer change. This distinction matters operationally because directed selection may produce large, consequential changes to deployment behavior; distinguishing those changes from mutation events is necessary for the governance workflows for the two evolution mechanisms to remain separate and correctly scoped.

Detection event recording and governance workflow triggering. Detection events are recorded per A2.40 with provenance metadata covering: which mutation event type was detected, when the detection occurred, which detection mechanism fired, and what triggered the detection. This makes the detection event the first entry in the mutation governance event chain — subsequent governance records in B2.63, B2.64, and B2.65 are traceable back to it. Upon detection, the mutation governance workflow is triggered in sequence: verification gate evaluation per B2.63 first, routing consideration per B2.64 second, pinning review per B2.65 third. Detection is the entry point; all subsequent governance steps depend on detection having fired. A mutation event that goes undetected integrates into the deployment without governance activation.

3. What makes mutation detection architecturally distinctive

Conventional AI architectures typically lack explicit mutation detection as an architectural commitment. In practice, model updates are noticed through operational observation — behavior changes, user reports, performance shifts — or through forced vendor migrations. There is no dedicated detection layer that actively monitors for mutation events as a class of architectural event, classifies them against a governed definition, records them with provenance, and triggers a governed workflow. Model updates happen reactively: after someone notices, or when the vendor compels.

The CKS treatment is structurally different. Detection rules are substrate-resident authoritative content. The substrate actively monitors the instinct layer through multiple mechanisms configured per deployment context. The mutation-vs-directed-selection distinction is applied at the detection layer, not post hoc. Detection events are recorded and serve as the entry point to the full governance workflow. This architecture is consequential: coverage determines whether governance activates. A deployment with incomplete detection coverage does not merely have incomplete monitoring; it has partial mutation governance, because mutation events that detection misses do not enter the workflow.

The treatment of detection rules as governed substrate content is the architectural property that distinguishes CKS mutation detection from monitoring tooling applied around AI systems. Monitoring tooling is external configuration; CKS detection rules are substrate content under the same authority architecture that governs every other aspect of the deployment. Detection coverage is itself subject to governance: humans can inspect, modify, and override detection rules, and detection rule authoring is subject to the same authority requirements as orchestration rule authoring per A2.04.

4. The biological analog as conceptual scaffold

Biology offers a structural analog in DNA repair mechanisms — cellular machinery that detects DNA damage, replication errors, and strand breaks, responding with correction, signaling, or apoptosis. The structural parallel to CKS mutation detection is: substrate-resident detection rules monitor the instinct layer

for specific change patterns; detection triggers a governed response workflow. The analog functions as conceptual scaffold; the architectural substance is independent of it.

Three points of divergence clarify where the analogy holds and where it does not. First, biological DNA repair conflates detection and response — the repair machinery that detects damage typically initiates the repair in the same molecular pathway. CKS separates detection and governance: detection fires the governance workflow, but the governance response is determined by humans through B2.63, B2.64, and B2.65. Detection and governance are architecturally distinct. Second, biological repair mechanisms cannot choose not to act on detected damage; repair is automatic. CKS mutation governance is governed — humans determine the response through subsequent steps. Detection activates governance; it does not determine the governance outcome. Third, CKS detection extends to behavioral drift that has no direct biological parallel, because CKS deployments face vendor behaviors — silent weight updates within stable version labels — that biological systems do not encounter.

5. Inherited Paper 1 commitments

Five Paper 1 commitments are directly load-bearing for B2.62's specification.

A1.08 (substrate-as-source-of-truth): The current LLM version is substrate content. Without this commitment, version monitoring has no authoritative baseline to detect discrepancy against. A1.08 is what makes the primary detection mechanism operational: the substrate records the current LLM version as authoritative state, and monitoring detects deviation from that record.

A2.04 (rule authoring): Detection rules are human-authored. They are not generated or autonomously modified by the LLM; authoring is a governance moment per A1.01 at which humans commit to detection coverage. This commitment holds detection rule authoring to the same standard as orchestration rule authoring.

A2.46 (the two-axis extension structure, Category 4): Detection rules are authoritative substrate content. They carry the authority properties of authoritative content: inspectable, modifiable under the three governance rights, versioned, and addressable. This is what distinguishes detection rules from external monitoring configuration.

A2.40 (six provenance metadata fields): Detection events are recorded with the full six-field provenance: what was detected, when, by which mechanism, and what triggered the classification. The detection event record is the root provenance entry for the mutation governance event chain; subsequent governance records are traceable to it.

A1.01 (governance): Detection rule configuration is governed. The detection rules themselves — which mechanisms are active, what thresholds trigger behavioral drift classification, how vendor notification monitoring is configured — are substrate content subject to human governance. Detection coverage is a governed operational decision, not a fixed architectural property.

6. Operational implications

Deployments configure detection rules per operational context. High-stakes deployments may activate all four detection mechanisms, configure low behavioral drift thresholds, and schedule frequent periodic reviews. Lower-stakes deployments may rely primarily on version monitoring and vendor notification integration, with less frequent periodic review and higher drift thresholds. Coverage depth is a design choice

governed by operational risk tolerance.

Detection coverage gaps are operational risks with a specific consequence: mutation events that fall in coverage gaps integrate into the deployment without governance activation. This makes coverage gap analysis a design discipline. The specific gap that behavioral drift detection addresses — silent weight changes within the same version label — illustrates the class of risk: a vendor may update model behavior without updating the version identifier, making version monitoring insufficient as the sole mechanism.

Detection rules are testable per the rule-authoring test (A5.04): a detection rule that cannot be shown to fire under a simulated mutation event of the type it claims to cover is not providing the coverage it claims. Testability of detection rules is therefore an operational verification that detection coverage holds. Periodic review provides detection coverage independent of automated monitoring — governance-scheduled reviews that examine whether the currently recorded LLM version is still current do not depend on any automated mechanism having fired; they catch mutation events automated monitoring missed.

Where LLM vendor notification systems publish deprecation notices and migration requirements, deployment detection configuration integrates those channels under the vendor notification monitoring mechanism. In composition contexts where the LLM is shared across composition partners per A2.47, a mutation event in the shared LLM is a mutation event for all deployments sharing it; detection configuration in composition contexts accounts for this scope.

Detection rules evolve through directed selection per B1.14. As deployments accumulate operational experience — false positives, missed mutation events, drift thresholds that prove miscalibrated — detection rules are refined through the standard authority architecture. The detection layer improves as better detection patterns emerge; it is not a fixed configuration installed at deployment birth.

7. Limits

Detection does not prevent mutation events. Mutation events occur in the vendor's domain — LLM weight changes, model deprecations, infrastructure migrations. The substrate has no authority over the vendor's actions. Detection operates after the mutation event has occurred or is occurring; it cannot intercept changes in the vendor's domain before they affect the instinct layer.

Detection does not equal governance. Detection triggers the mutation governance workflow, but governance must proceed through B2.63, B2.64, and B2.65. A deployment in which detection fires but the subsequent workflow steps are not executed has detected a mutation event without governing it. Detection is necessary but not sufficient for mutation governance.

Detection coverage is not guaranteed complete. The four detection mechanisms provide depth but not exhaustive coverage. A mutation event involving neither an explicit version change, a published vendor notification, a behavioral shift above drift thresholds, nor a discrepancy caught by periodic review may go undetected. Coverage is operationally important and imperfect.

Behavioral drift detection is supplementary, not primary. Version monitoring is the primary mechanism. Behavioral drift has multiple causes — DNA-layer changes, input pattern shifts, substrate content modifications — that are not mutation events. Using behavioral drift as the primary mechanism would produce false positives for non-mutation events; it functions correctly as supplementary coverage for silent weight changes that version monitoring cannot detect.

Detection does not classify mutation events as harmful. Harmfulness, integration risk, and appropriate governance response are determined through the governance workflow — specifically through the verification step per B2.63. Detection classifies a change as a mutation event; it does not determine whether that mutation event is beneficial, neutral, or harmful in the deployment context.

Detection rules are not static. They are governed substrate content that evolves. Detection coverage at any operational stage reflects the current state of detection rule authoring; coverage may improve, regress, or shift focus as directed selection acts on detection rules over time.

8. Operational test

A CKS deployment instantiates the mutation detection operational specification if and only if all of the following hold:

1. Detection rules are authored by humans per A2.04 and carried as authoritative substrate content per A2.46.
2. At minimum, LLM version monitoring is active as the primary detection mechanism, with the current LLM version recorded in the substrate per A1.08 and monitored for discrepancy.
3. At least one backup detection mechanism — vendor notification monitoring, periodic review, or behavioral drift detection — is active and configured to cover mutation event types that version monitoring may miss.
4. Detection applies the B2.61 instinct-layer test: DNA-layer changes through directed selection are classified as non-mutation events and do not trigger the mutation detection workflow.
5. Detection events are recorded per A2.40 with provenance metadata sufficient to identify the mutation event type, detection time, mechanism, and trigger.
6. Detection triggers the mutation governance workflow in the sequence B2.63 → B2.64 → B2.65.

A deployment that fails any of (1)–(6) may detect some mutation events through ad hoc means, but does not instantiate the mutation detection operational specification as formalized here.

9. Naming as standalone and position in the B1.13 decomposition

The six-note B1.13 decomposition requires mutation detection to be formalized as a standalone step. Subsumption into the mutation event definition (B2.61) would conflate two distinct architectural commitments — what counts as a mutation event, and how deployments detect such events — that have separately patentable territory. Subsumption into the verification gate treatment (B2.63) would obscure the prior-art territory of the detection architecture itself, treating detection as a mere precondition rather than as an independently specifiable operational commitment.

The detection step is architecturally distinctive because it is the point at which events in the vendor’s domain cross into the substrate’s governance domain. A mutation event as B2.61 defines it is a fact about what the LLM does; a detection event as B2.62 formalizes it is a substrate record of that fact being identified and classified. The boundary between vendor domain and substrate domain is crossed at detection. This is the architectural work that the detection specification does.

B2.62 precedes B2.63's verification gate triggering, B2.64's routing adaptation, and B2.65's pinning enforcement. All three subsequent steps in the B1.13 decomposition depend on detection having fired. B2.66 will close the decomposition with mutation governance verification — the operational check that the full governance chain executed correctly once detection triggered it. Following B2.66, Phase B2 continues with the B1.14 directed selection decomposition, carrying the evolutionary dynamics analysis through horizontal and vertical evolution.

Source papers

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Mutation Detection Operational Specification: Substrate-Resident Detection Rules, Coverage Depth, and Governance Workflow Triggering in CKS Deployments*. May 12, 2026. ORCID: 0009-0004-8065-3235.